
《算法设计与分析》 课程内容 回顾



课程内容及教学概述

□ 第一部分 基础知识

第一章 算法在计算中的作用

第二章 算法入门

第三章 函数的增长

第四章 递归式

□ 第二部分 排序和顺序统计学

第七章 快速排序

第八章 线性时间排序

第九章 中位数和顺序统计学



课程内容及教学概述

□ 第四部分 高级设计和分析技术

第十五章 动态规划

第十六章 贪心算法



第一部分 基础知识



第一章 算法在计算中的作用

□ 内容提要

■ 1.1 算法

■ 1.2 作为一种技术的算法

□ 什么是算法？研究算法的目的？



第一章 算法在计算中的作用

□ 1.1 算法

- 简单来说，**算法**（**algorithm**）就是定义良好的**计算过程**，
取一个或一组值作为输入，
并产生一个或一组输出。
- 也就是，算法是一系列的**计算步骤**，
用来将输入数据转换成输出结果。

1.1 算法

- 例如，**排序问题**描述如下。
 - **输入**：由n个数据元素构成的序列 $\langle a_1, a_2, \dots, a_n \rangle$.
 - **输出**：对输入序列的排序 $\langle a'_1, a'_2, \dots, a'_n \rangle$ ，使得 $a'_1 \leq a'_2 \leq \dots \leq a'_n$.
- **排序**是基本的操作，有许多好的算法。
- 排序算法的**衡量因素**很多，涉及到：
 - 数据的项数、
 - 已经排好的程度、
 - 对数据项取值可能的限制、
 - 存储设备的类型（内存、磁盘、磁带）等

1.1 算法

- **算法的正确性**：如果一个算法对每个输入都能得到正确的结果，并停止，则称为是**正确的**。
 - **不正确的算法**对某些输入来说，可能不会停止，或得到的不是预期的结果。
 - 有时，如果算法的**错误率**可以得到**控制**的话，有时也是有用的。
- **算法的描述**：自然语言、计算机语言等。
 - **要求**：算法的规格说明必须提供关于代码运行的计算过程的精确描述。

1.1 算法

- 各领域的例子形式上存在较大差异，但其底层所涉及到的支撑技术具有许多共性。
- 许多有趣算法的**两个共同特征**:
 - 有很多候选的解决方案，其中大多数都不是所需要的。
找到真正需要的解决方案往往不容易。
 - 有实际的应用。
最短路径：运输公司成本；互联网中：路由节点
- **数据结构**
 - 算法必然涉及到数据结构：数据的组织方式。
 - 不同特性和应用场合。
- 算法设计和分析技术
 - 面临**新的问题时**所需要的技术



算法设计的思路:面临新的问题时所需要的技术

□ 算法设计过程:

- 实际问题描述 -> 问题建模/问题形式化
- ⇒ 现有解决方法调研
- ⇒ 解决思路: 以一个小规模例子分析现有方法存在的问题, 以及解决该问题的思路
- ⇒ 伪代码: 通过小规模例子模拟求解过程, 写出求解步骤
- ⇒ 通过几个例子验证伪代码的正确性
- ⇒ 算法分析: 分析时间复杂度、以及是否有进一步的改进空间
- ⇒ 代码实现
- ⇒ 实验结果分析与代码修改、优化
- ⇒ 实验结果记录、文档撰写



1. 针对一个整数 n ，把从1到 n 的数字无重复的排列成环，且使每相邻两个数（包括首尾）的和都为素数，则称为素数环。

要求：

- （1）请选择合适的数据结构对该问题进行建模，并给出建模的基本思路。
- （2）可使用什么算法求解该问题？请给出求解该问题的基本思想。
- （3）请给出算法的伪代码，并在重要步骤上给出注释。



1.1 算法

- **P类问题**：存在多项式时间算法的问题。(P: polynomial, 多项式)
- **NP类问题**：能在多项式时间内验证得出一个正确解的问题。(NP: Nondeterministic polynomial, 非确定性多项式)
 - P类问题是NP问题的子集，因为存在多项式时间解法的问题，总能在多项式时间内验证
- **NPC类问题**（NP完全问题，Nondeterminism Polynomial complete）：存在这样一个NP问题，所有的NP问题都可以约化成它。换句话说，只要解决了这个问题，那么所有的NP问题都解决了
 - 定义：1) 首先，是一个NP问题；2) 其次，所有的NP问题都可以约化到它
- **NP难问题**（NP-hard问题）：满足NPC问题定义的第二条但不一定要满足第一条
 - 不一定是NP问题。即使NPC问题发现了多项式级的算法，NP-Hard问题有可能仍然无法得到多项式级的算法



1.1 算法

□ 关于NP完全问题

-----有趣的问题:

- 迄今为止，**没有找出**NP完全问题的有效解法，
但也**没有人能证明**NP完全问题的有效解法是不存在的。
- 如果NP问题中的任何一个问题存在有效算法，
则该集合中其他所有问题都存在有效算法。
- 有几个NP完全问题类似于（但又不完全同于）一些有着已知有效算法的问题，对一个陈述的一点小小改动，就会对其一直最佳算法的效率带来很大的变化。



1.1 算法

◆ 如何识别NP完全（NPC）问题

- 元素较少时算法的运行速度非常快，但随着元素数目的增加，速度会变得非常慢。
- 涉及“所有组合”的问题通常是NP完全问题。
- 不能采用分治法的思想将大问题分解成小问题，必须考虑各种可能情况，这很可能是NP完全问题。
- 如果问题涉及“序列”或“集合”，并难以解决，很可能就是NP完全问题。

□ 严格证明的思路：先证明它至少是一个NP问题,再证明其中一个已知的NPC问题能约化到它



第三章 算法的渐进运行时间

□ 分析的类型

最坏一情况：（通常）

- $T(n)$ = 任何输入大小为 n 的情况下算法的最大运行时间

平均一情况：（有时）

- $T(n)$ = 所有输入大小为 n 的情况下算法的期望运行时间
- 需要假设输入的统计分布

最好一情况：（骗人的）

- 即使一个很慢的算法在某些输入下也会运行很快

不依赖于计算机的时间

插入排序最坏情况的运行时间是多少？

□ 它依赖于计算机的速度：

- 相对速度（在同一计算机上）
- 绝对速度（在不同计算机上）

□ 好主意：

- 忽略计算机相关的常数
- 看 $n \rightarrow \infty$ 时 $T(n)$ 的增长。

（问题的规模！）

“渐进分析”

一 记号 Θ （渐进确界）

数学：

$\Theta(g(n)) = \{f(n) : \text{存在正常数 } c_1, c_2 \text{ 和 } n_0, \text{ 使得}$
 $\text{对所有的 } n \geq n_0, \text{ 有 } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$

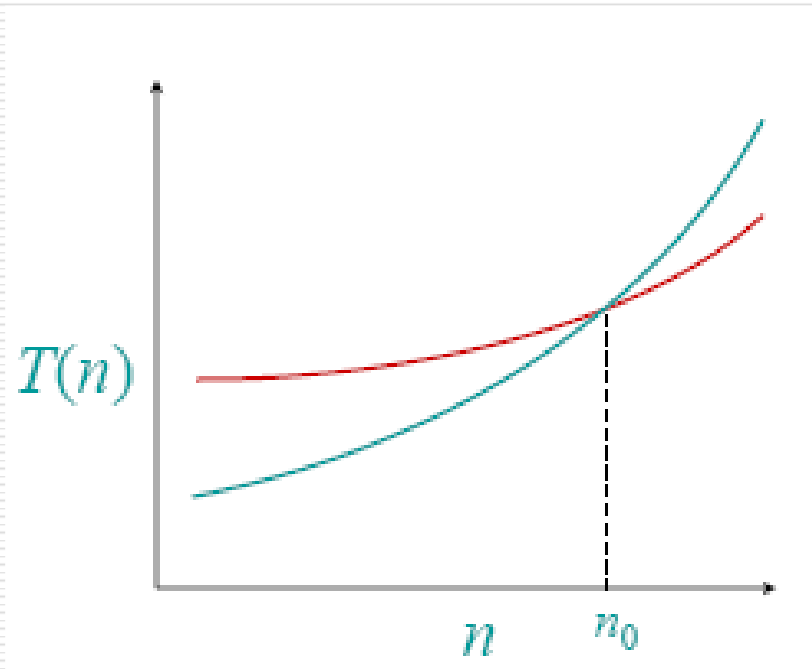
工程：

- 丢弃低阶项，忽略首项的常数
- 例子： $3n^3 + 90n^2 - 5n + 6046 = \Theta(n^3)$



渐进性能

- 当 n 充分大的时候, $\Theta(n^2)$ 算法总是比 $\Theta(n^3)$ 算法好



- 我们不应该忽略渐进比较慢的算法
- 真实的设计环境通常要在工程目标上作仔细的权衡
- 渐进分析是帮助我们思考的有效工具

渐进记号

- Θ 记号 渐进确界
- O 记号 渐进上界
- Ω 记号 渐进下界
- o 记号 非渐进紧确的上界
- ω 记号 非渐进紧确的下界



第四章 求解递归式

- 当算法递归调用时，运行时间通常可以用递归式来表示。
- 例如 Merge_sort算法的时间函数 $T(n)$:

$$T(n) = \begin{cases} O(1) & n=1 \\ 2T(n/2) + O(n) & n>1 \end{cases}$$

解为 $O(n \log n)$

如何求解？

代换法，递归树方法，主方法



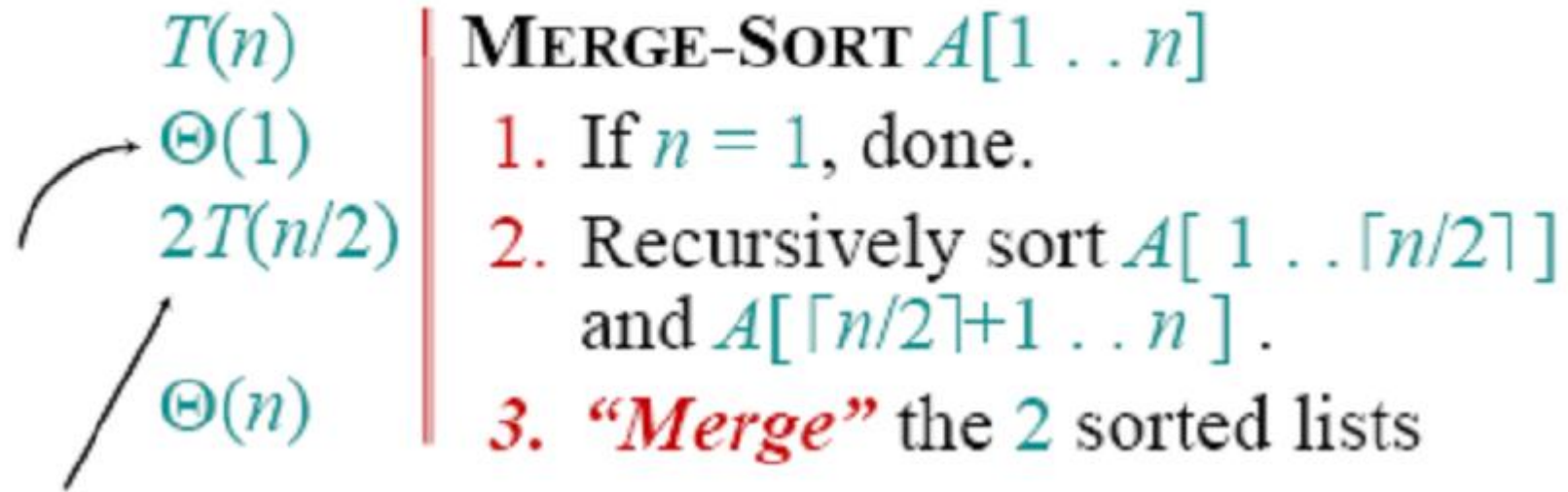
1. 算法分析中，记号 O 表示_____，记号 Θ 表示_____。

A. 渐进下界 B. 渐进上界 C. 非紧上界 D. 紧渐进界 E. 非紧下界



分析合并排序

不严谨



此处不严谨: 应该是 $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$

但是对渐进分析没有影响

合并排序的递归表示

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1; \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

- 当 n 足够小的时候，我们通常忽略起始状态 $T(n) = \Theta(1)$ ，但仅仅是在它对递归的渐进分析的结果没有影响的时候。
- 找 $T(n)$ 上界的方法？

求解递归式的方法

- 代换法
- 递归树
- 主方法



4.1 代换法

- 代换法求解的两个步骤：
 - (1) 猜测解的形式
 - (2) 用数学归纳法找出使得解真正有效的常数
- 只能用于容易猜测的情形。
- 例，确定递归式 $2T(n/2) + n$ 的解

为此，先猜测 $T(n)=O(n\log n)$ ，
然后证明 $T(n) \leq c n \log n$ (c 是某个常数)
先假设：对 $n/2$ 成立，即 $T(n/2) \leq cn/2 \log(n/2)$ ，
则 $T(n)=2T(n/2) + n$
 $\leq 2(cn/2 \log(n/2)) + n$
 $\leq cn \log(n/2) + n$
 $= cn \log n - cn \log 2 + n$
 $\leq cn \log n$

问题：取决于猜测



4.2 递归树

求解

$$T(n) = 2T(n/2) + cn, \forall c > 0。$$

递归树

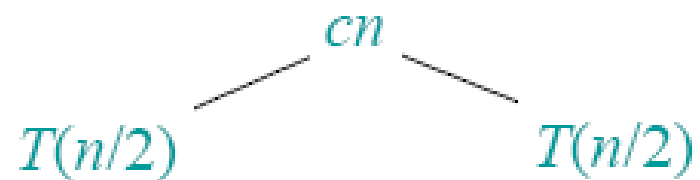
求解

$$T(n) = 2T(n/2) + cn \quad \text{且 } c > 0。$$
$$T(n)$$

递归树

求解

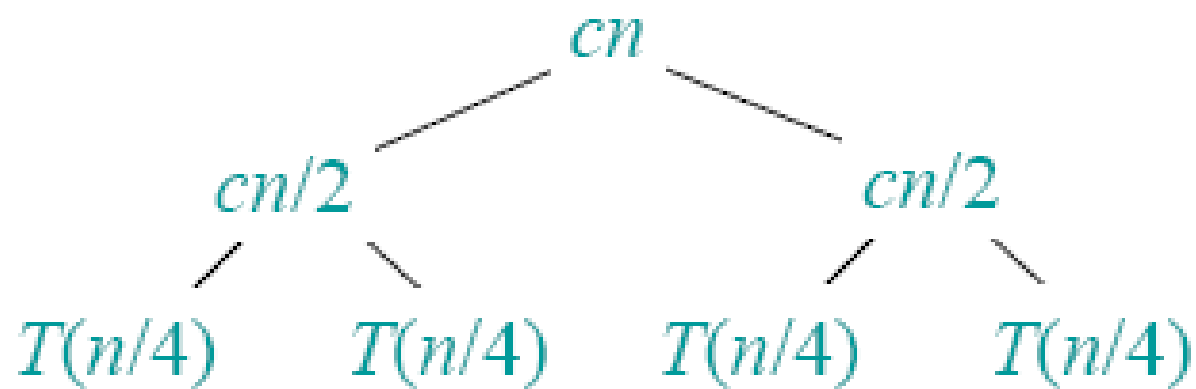
$$T(n) = 2T(n/2) + cn, \text{ 其中 } c > 0。$$



递归树

求解

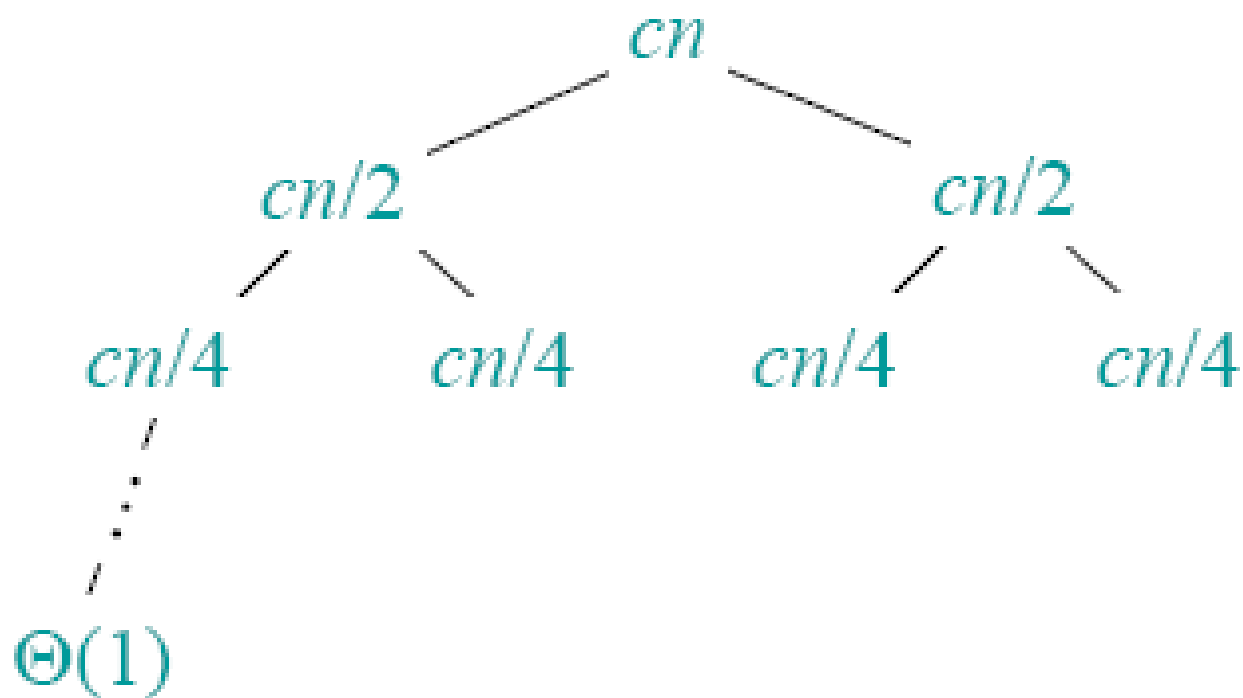
$$T(n) = 2T(n/2) + cn, \text{ 其中 } c > 0。$$



递归树

求解

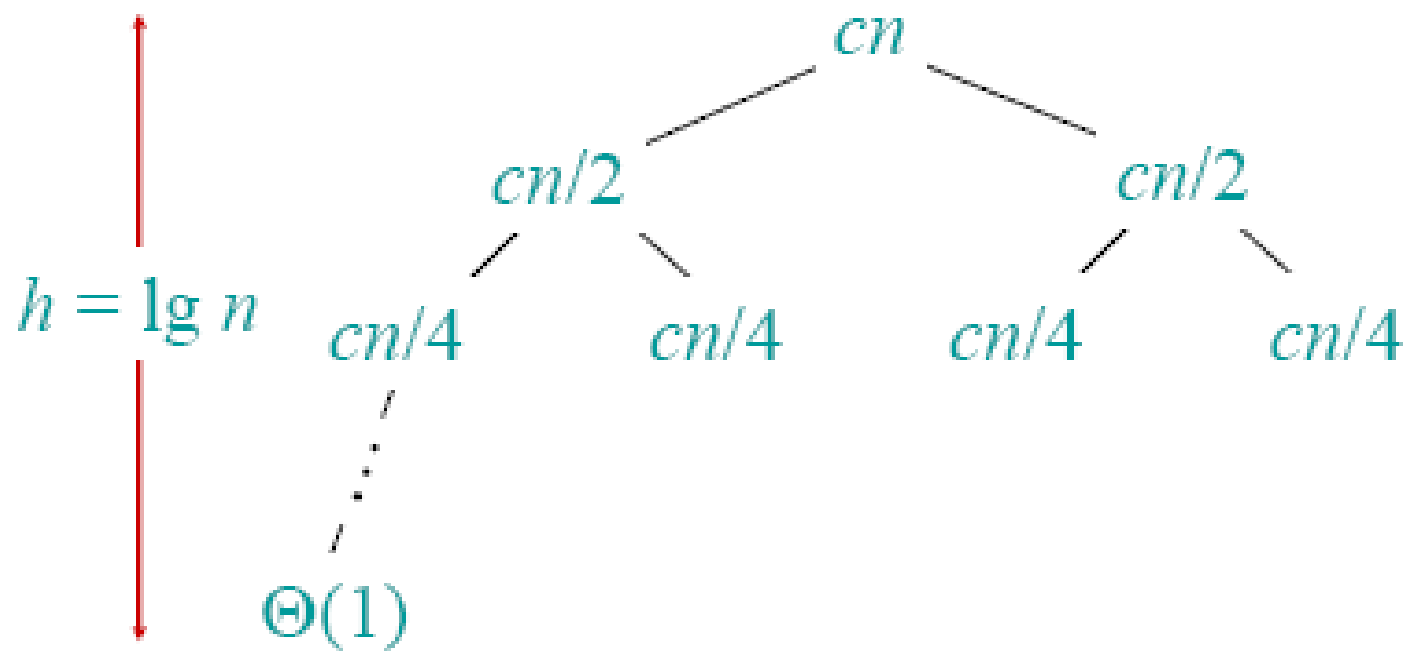
$$T(n) = 2T(n/2) + cn \quad \text{其中 } c > 0。$$



递归树

求解

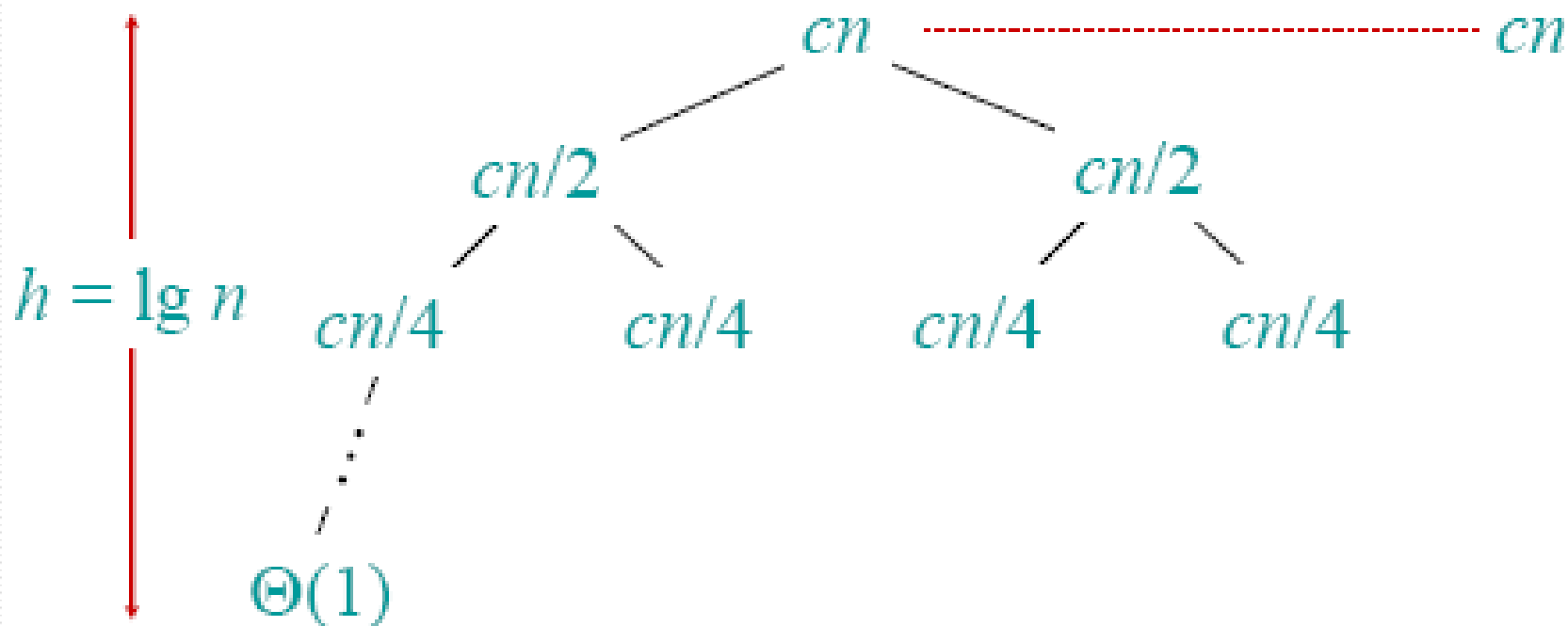
$$T(n) = 2T(n/2) + cn \quad \text{且 } c > 0。$$



递归树

求解

$$T(n) = 2T(n/2) + cn \quad \text{其中 } c > 0 \text{ 为常数。}$$

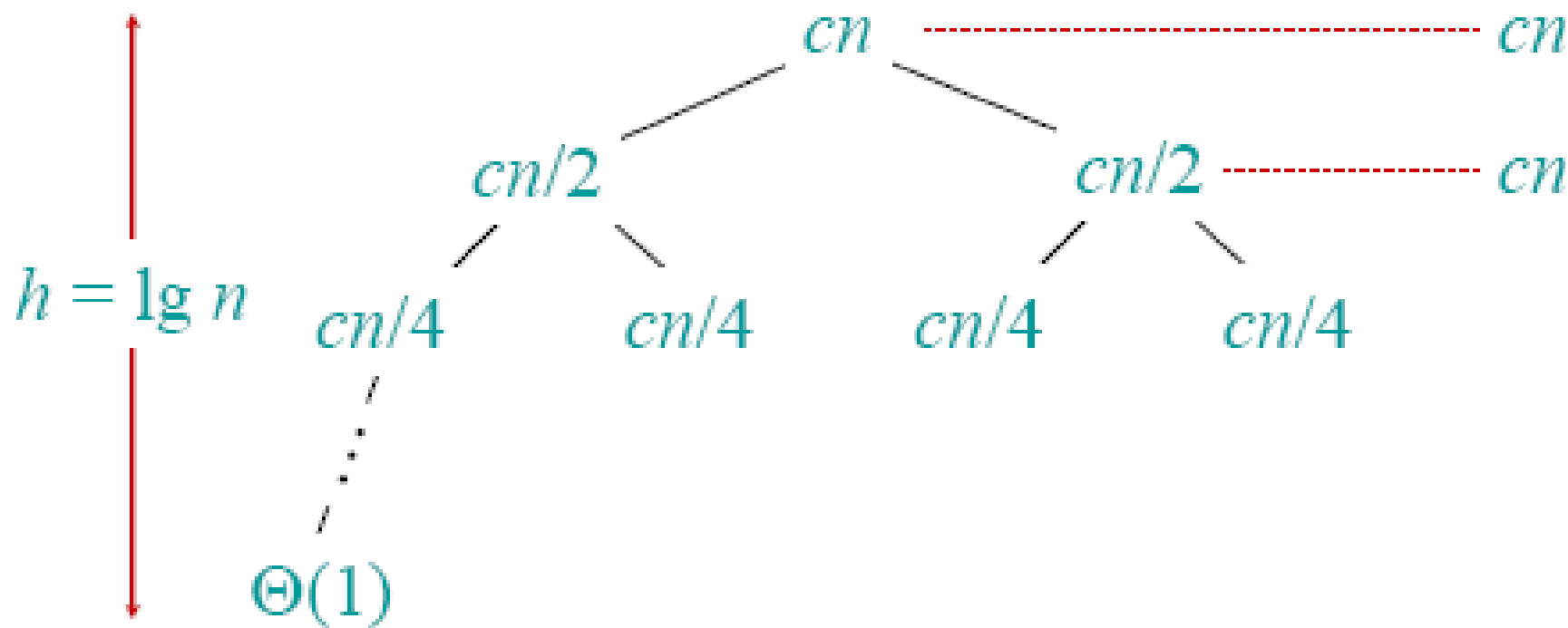


递归树

求解

$$T(n) = 2T(n/2) + cn \quad c > 0$$

, 且为常数。

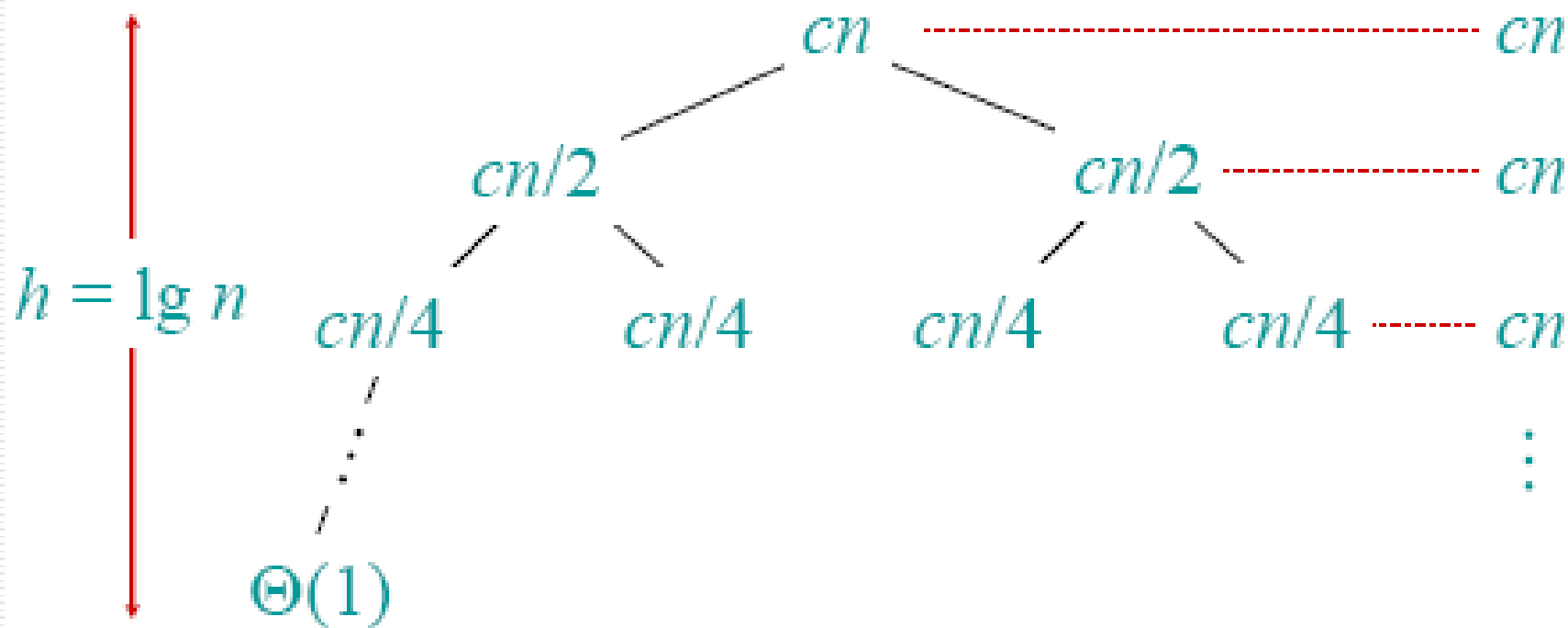


递归树

求解

$$T(n) = 2T(n/2) + cn \quad c > 0$$

, 且为常数。

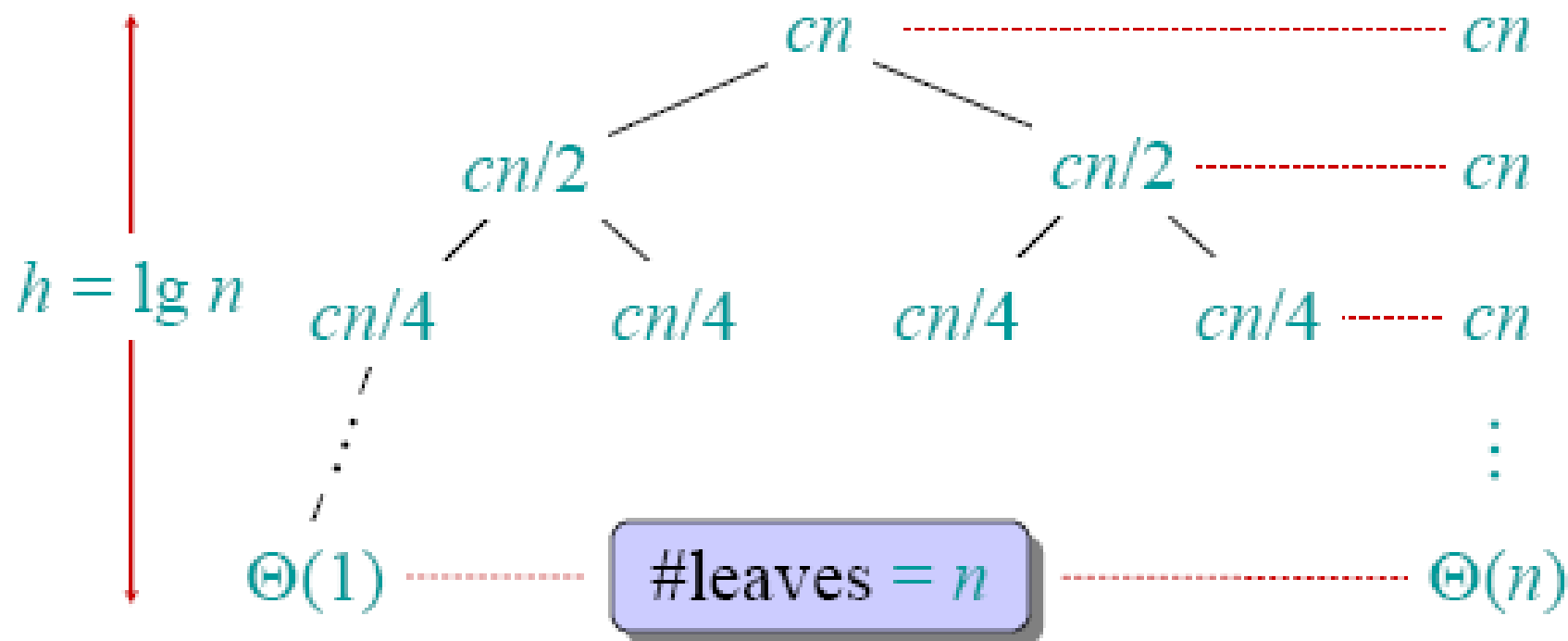


递归树

求解

$$T(n) = 2T(n/2) + cn \quad c > 0$$

, 且为常数。

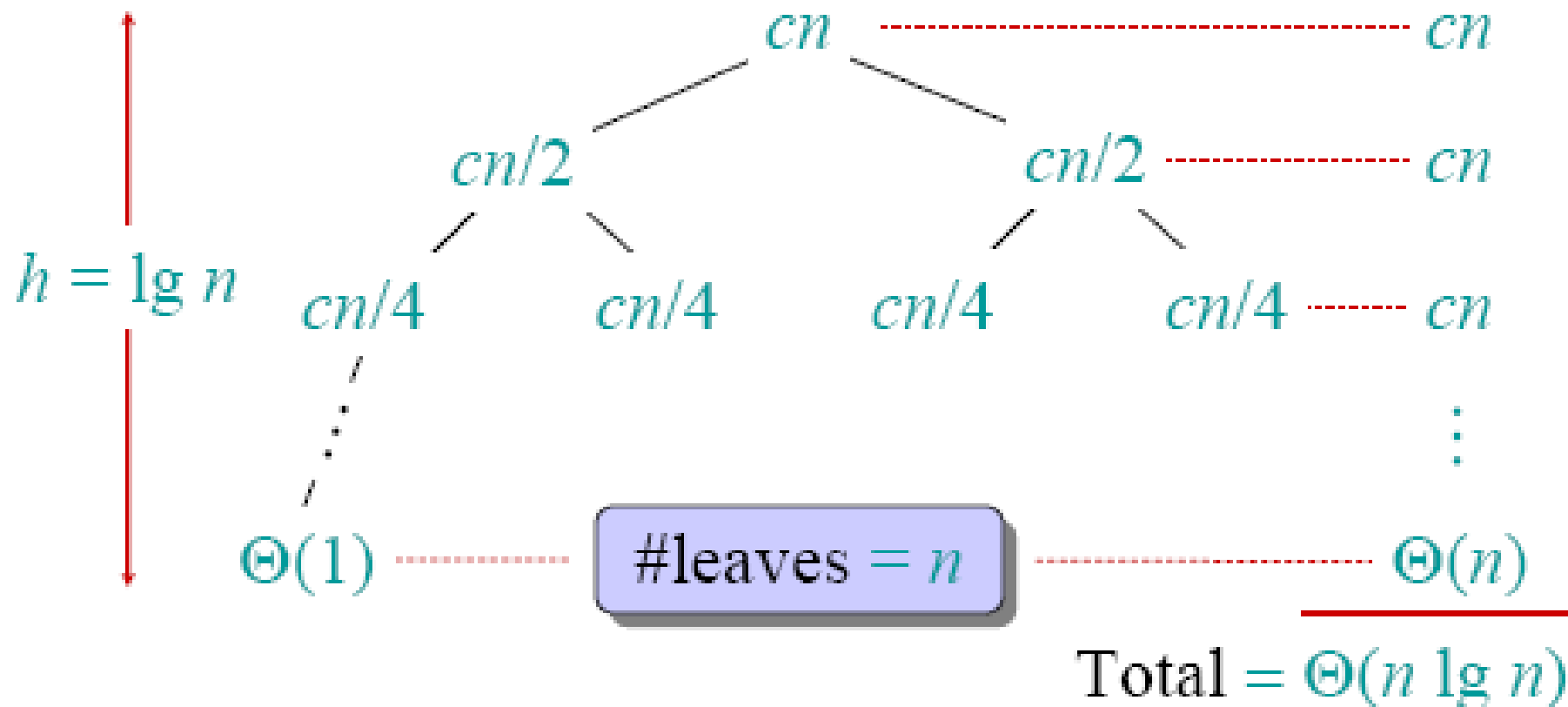


递归树

$$T(n) = 2T(n/2) + cn \quad c > 0$$

c 为常数

求解



递归树方法

- 递归树对算法递归执行的花费（时间）建模
- 递归树方法可以用作代换法之前的猜想。
- 递归树方法可能不是很可靠
- 然而，递归树方法有启发的作用



4.3 主方法

主方法（master method）

□ 主方法给出求解如下形式的递归式方法：

$$T(n)=aT(n/b)+f(n)$$

其中， $a \geq 1$, $b > 1$ 是常数

$f(n)$ 是渐近正的函数。

描述了将规模为 n 的问题划分为 a 个子问题的算法的运行时间，每个子问题的规模为 n/b 。

每个子问题的求解时间为 $T(n/b)$,

划分和合并的时间为 $f(n)$.



主方法

主方法适用于下面的递归形式

$$T(n) = aT(n/b) + f(n),$$

$a \geq 1, b > 1$, f 为正的渐进函数



三种情况

比较 $f(n)$ 和 $n^{\log_b a}$:

□ 1. $f(n) = O(n^{\log_b a - \varepsilon})$, 常量 $\varepsilon > 0$ 。

$f(n)$ 比多项式 $n^{\log_b a}$ 增长的慢（慢 n^ε 倍）

解: $T(n) = \Theta(n^{\log_b a})$.

2. $f(n) = O(n^{\log_b a})$

解: $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$



三种情况（续）

3. 比较 $f(n)$ 和 $n^{\log_b a + \varepsilon}$:

$f(n) = \Omega(n^{\log_b a + \varepsilon})$, 常量 $\varepsilon > 0$

$f(n)$ 增长比多项式 $n^{\log_b a}$ 快（快 n^ε 倍）

并且 $f(n)$ 满足规定条件

$af(n/b) \leq cf(n)$, 常量 $c < 1$ 。

解: $T(n) = \Theta(f(n))$ 。



举例

Ex. $T(n) = 4T(n/2) + n$

请求解递归式 $T(n)=3T(n/2)+n$ 。

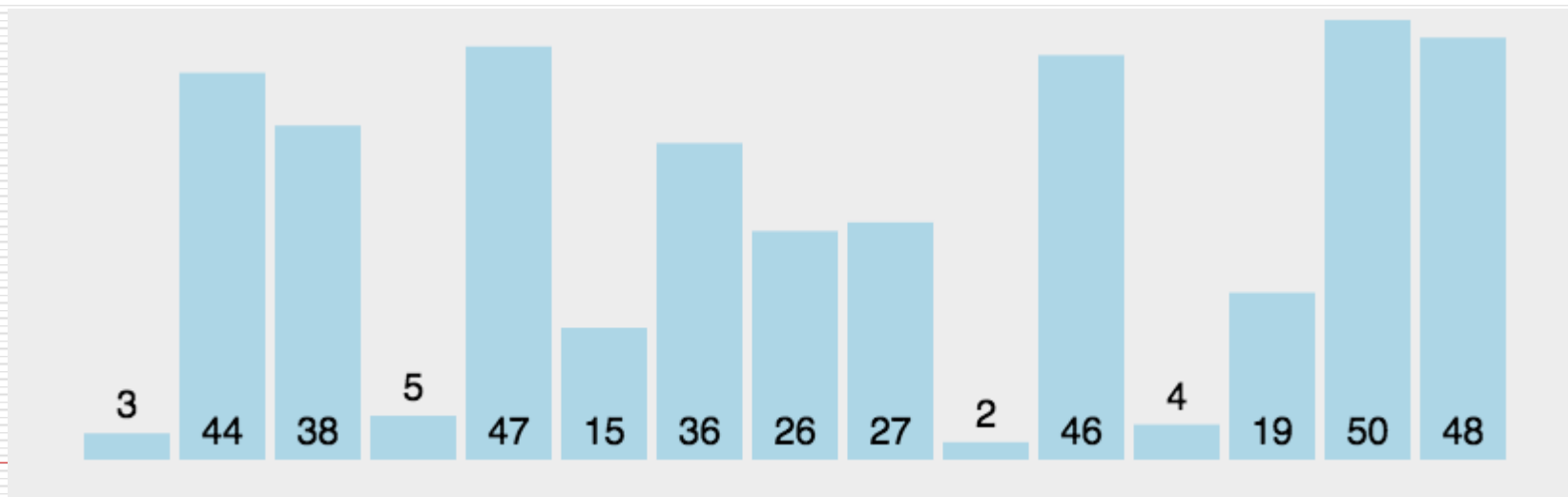


第二部分 排序和顺序统计学



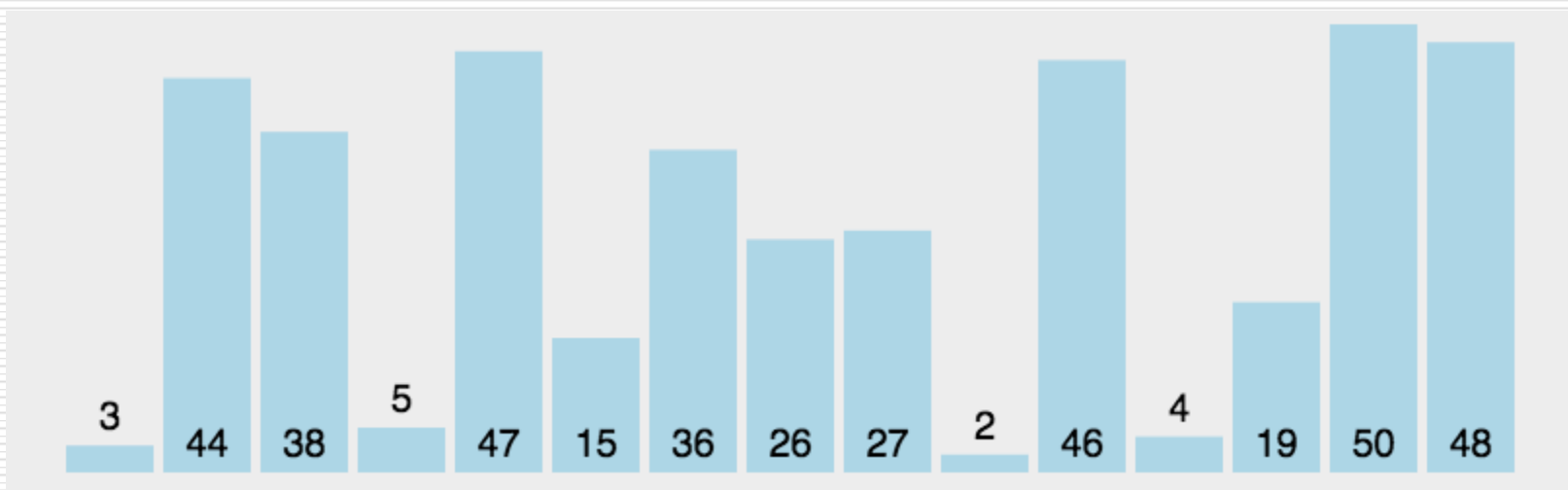
冒泡排序

- 比较相邻的元素。如果第一个比第二个大，就交换他们两个。
- 对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对。这步做完后，最后的元素会是最大的数。
- 针对所有的元素重复以上的步骤，除了最后一个。
- 持续每次对越来越少的元素重复上面的步骤，直到没有任何一对数字需要比较。



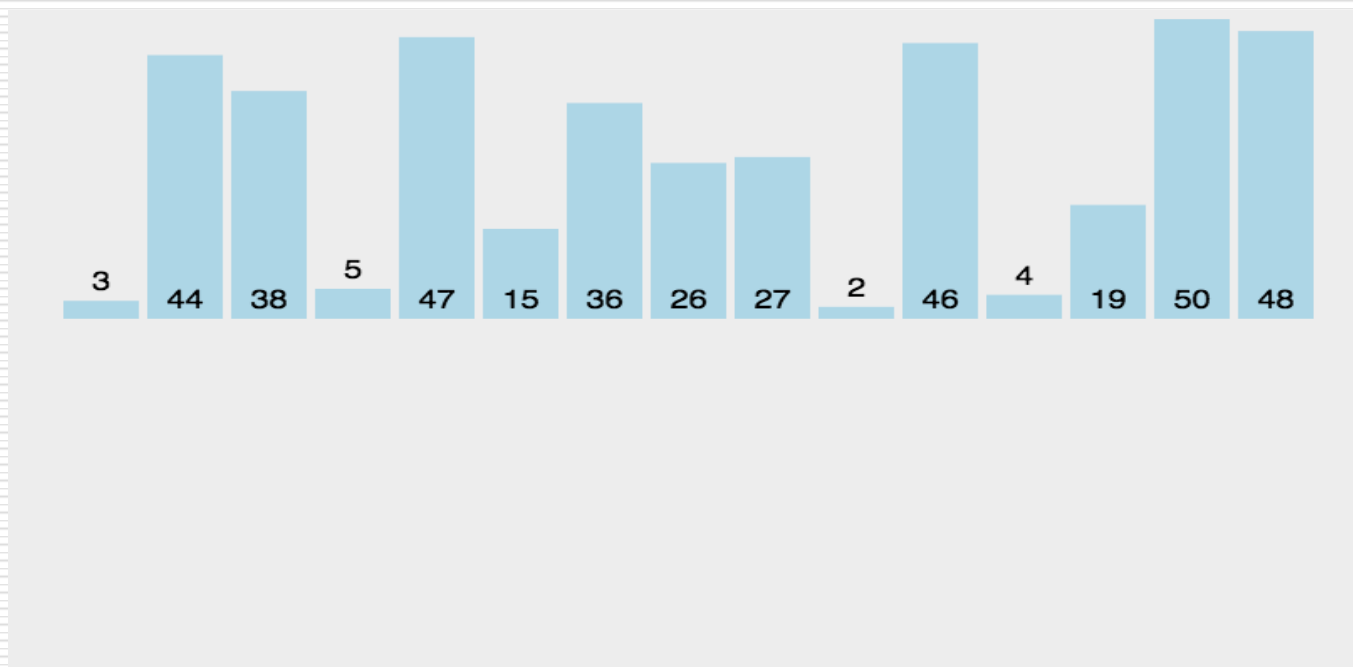
选择排序

- 首先在未排序序列中找到最小（大）元素，存放到排序序列的起始位置，然后，再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。以此类推。
- 问题：是否所有排序算法的最好时间复杂度都是 $O(n)$ ？



插入排序

- 通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。需要反复把已排序元素逐步向后挪位，为最新元素提供插入空间。



希尔排序

- 是直接插入排序的改进版本。把记录按下标的一定增量分组，对每组使用直接插入排序算法排序；随着增量逐渐减少，每组包含的关键词越来越多，当增量减至1时，整个文件恰被分成一组，算法便终止。
- 问题：与直接插入相比，时间复杂度为什么可以少于 $O(n^2)$ ？

@五分钟学算法之希尔排序



希尔能突破 $O(N^2)$ 的界，可以用逆序数来理解。

排序的本质就是消除逆序数。对于随机数组，逆序数是 $O(N^2)$ 的。

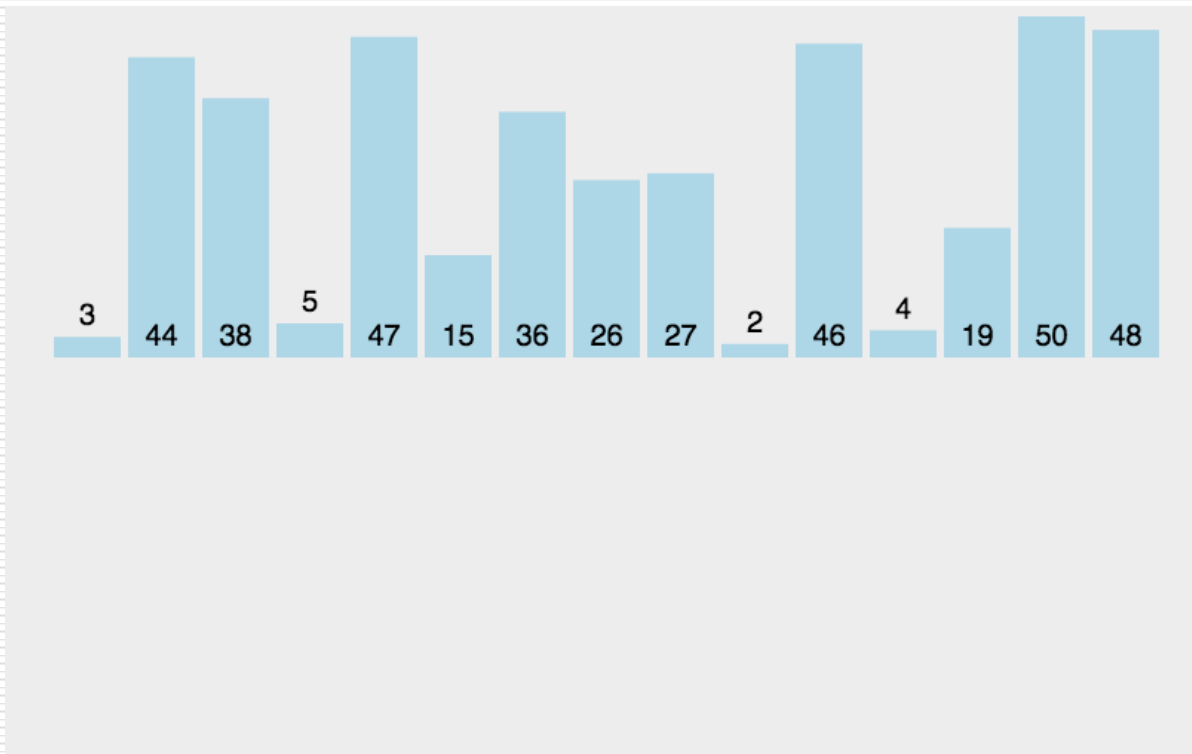
如果采用“**交换相邻元素**”的办法来消除逆序，每次正好只消除一个，因此必须执行 $O(N^2)$ 的交换次数，这就是冒泡、插入等算法只能到平方级别的原因

反过来，基于交换元素的排序要想突破这个下界，必须执行一些比较，**交换相隔比较远的元素**，使得一次交换能消除一个以上的逆序，希尔、快排、堆排等等算法都是交换比较远的元素。



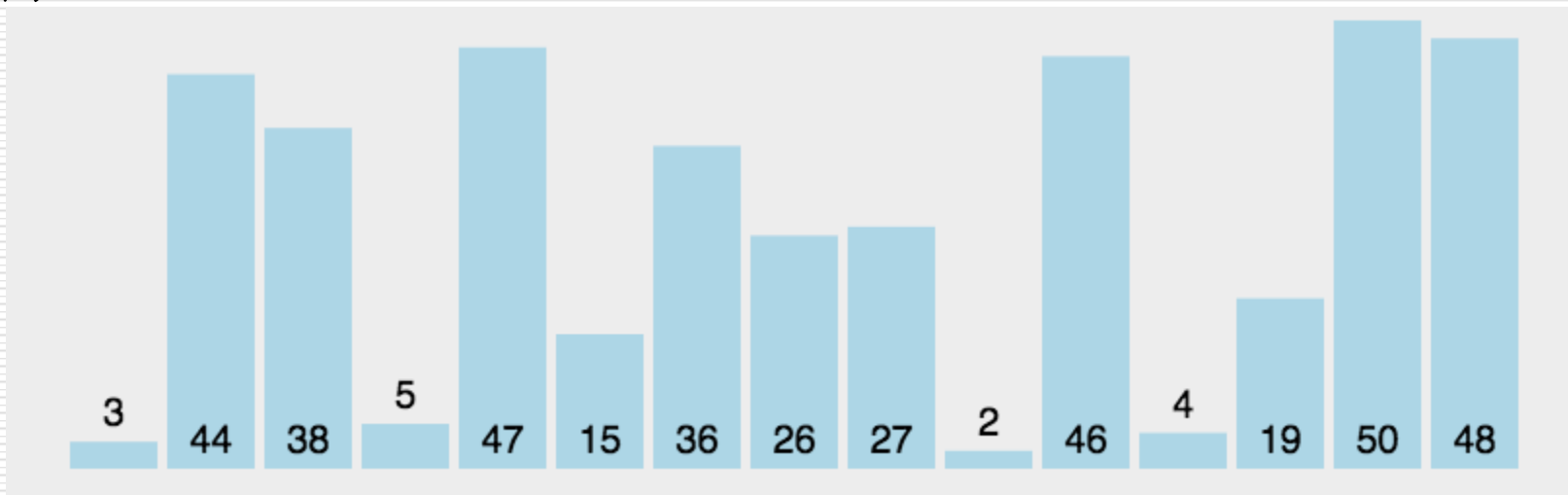
归并排序

分治法的基本思想：将已有序的子序列合并，得到完全有序的序列；即先使每个子序列有序，再使子序列段间有序



快速排序

基本思想：通过一趟排序将待排记录分隔成独立的两部分，其中一部分记录的关键字均比另一部分的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。



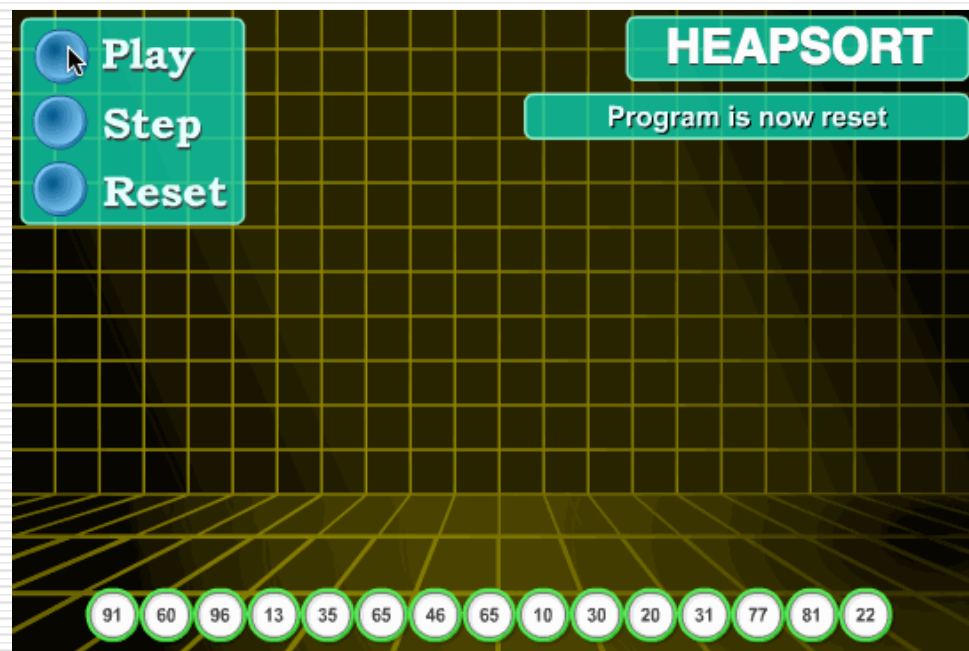
延伸

□ 求一个数组里面最小的 k 个数？



堆排序

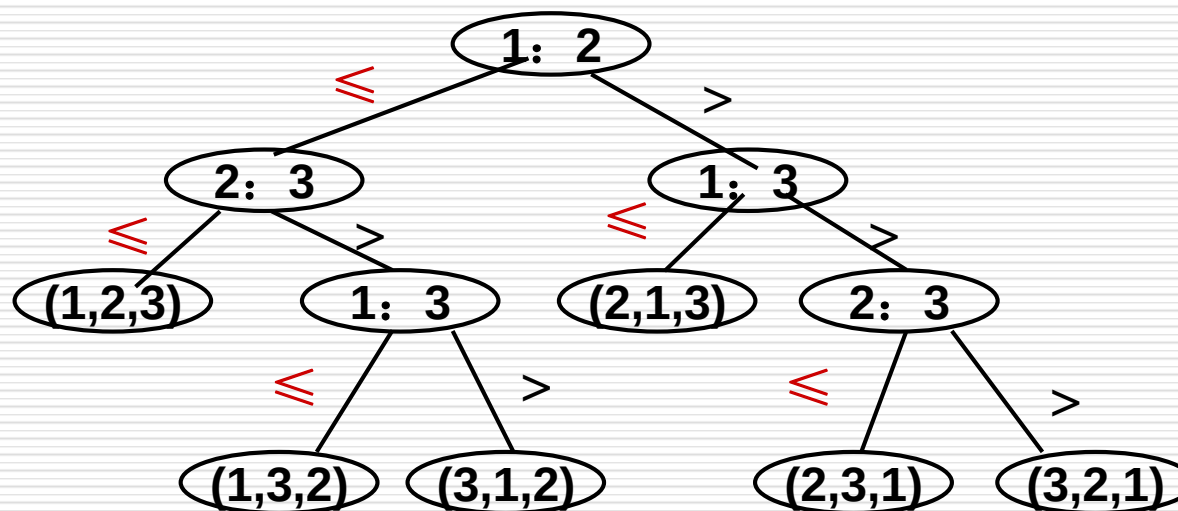
- 堆排序：利用堆这种数据结构所设计的一种排序算法。
- 堆是一个近似完全二叉树的结构，并同时满足如下性质：即子结点的键值或索引总是小于（或者大于）它的父节点。



基于比较的排序算法时间下界

□ 以3个元素的比较为例

- 内部节点：对应一个比较操作
- 分支：对应一次比较的一种情况
- 叶子节点：对应一个排列



排序算法时间下界

最坏情况下届

□ **定理：**任意一个对 n 个元素排序的决策树的高度为 $O(n \lg n)$ 。

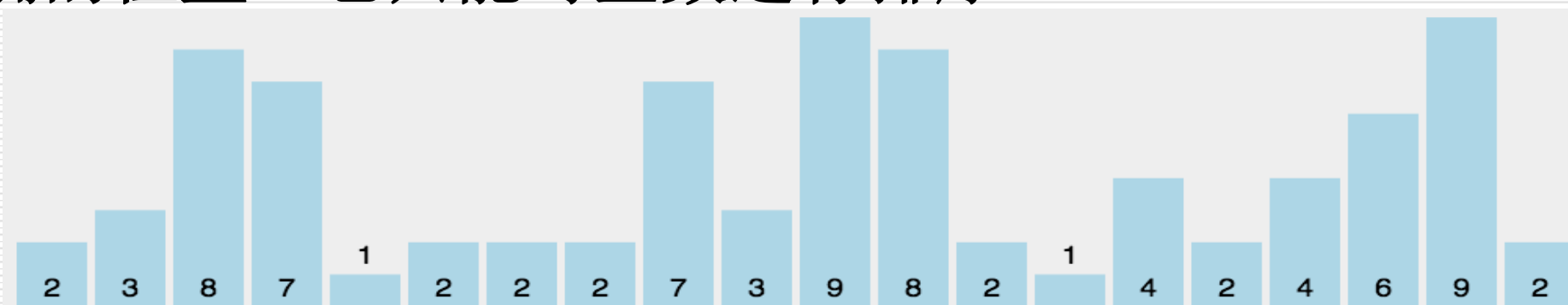
□ **证明：**

- 考虑一个高度为 h ，有 m 个可达叶子节点的决策树，
- 对应于对 n 个元素所做的比较排序。
- 因为所有可能的排列数为 $n!$ ，
- 每一种都对应决策树的一个叶子。
- 因此有 $n! \leq m$ 。
- 又由于高度为 h 的树的叶子数 $\leq 2^h$ ，
- $n! \leq m \leq 2^h$
- 因此， $h \geq \lg(n!) = O(n \lg n)$ 。



计数排序

- 计数排序使用一个额外的数组**C**，其中第*i*个元素是待排序数组**A**中值等于*i*的元素的个数。然后根据数组**C**来将**A**中的元素排到正确的位置。它只能对整数进行排序。



基数排序

- 基本思路：将所有待比较数值（正整数）统一为同样的数位长度，数位较短的数前面补零。然后，从最低位开始，依次进行一次排序。这样从最低位排序一直到最高位排序完成以后，数列就变成一个有序序列。

3	44	38	5	47	15	36	26	27	2	46	4	19	50	48
---	----	----	---	----	----	----	----	----	---	----	---	----	----	----

第四部分 高级设计和分析技术



□ 整数排序问题:

- (1) 请介绍计数排序的基本思想和伪代码。
- (2) 假设待排序序列为 $\{4, 2, 2, 8, 3, 3, 1\}$, 请给出使用计数排序对该数列进行升序排序的过程。



高级设计和分析技术

- 分治法
- 动态规划
- 贪心算法
- 回溯法
- 分支限界法



-
- 针对斐波那契数列

$$F(0)=0$$

$$F(1)=1$$

$$F(n)=F(n-1)+F(n-2)$$



采用递归算法求递归方程 $f(n)=f(n-1)+f(n-2)$ ，算法描述如下。

问：(1) 该算法的时间复杂度是多少？

(2) 有没有更好的算法来计算 $f(n)$?若有请描述该算法的基本思想。

```
1. int f(int n)
2. {
3.     if(n == 1)
4.         return 1;
5.     else if(n == 0)
6.         return 0;
7.     else
8.         return f(n - 2) + f(n - 1);
9. }
```



- **背包问题:** 给定 n 物体, 第 i 个 价值 v_i 元 并且重量 w_i 磅, 一个贼希望带走价值尽量多的东西, 但是他的背包仅仅可以容纳 W 磅.
 - **0-1 背包问题:** 每个物体要么带着, 要么放下 (0-1 决定).
 - **分数 背包问题:** 允许拿走部分物体.
-
- (1) 分数背包问题是否具有贪婪选择特征? 请给出证明。
 - (2) 若使用贪心算法求解分数背包问题, 请给出基本求解思路
 - (3) 请给出算法伪代码
 - (4) 0-1 背包问题是否具有贪婪选择特征?



钢条切割问题：Serling 公司购买长钢条，将其切割为短钢条出售。切割工序本身没有成本支出。公司管理层希望知道最佳的切割方案。假定我们知道 Serling 公司出售一段长为 i 英寸的钢条的价格为 $p_i (i=1,2,\dots)$ ，单位为美元)。钢条的长度均为整英寸。钢条切割问题是指给定一个长度为 n 英寸的钢条和一个价格表 $p_i (i=1,2,\dots,n)$ ，求切割钢条的方案，使得销售收益 r_n 最大。问：

- (1) 长度为 n 的钢条总共有多少种切割方案？请分析原因。
- (2) 请对该问题进行建模（给出该问题的形式化描述）。
- (3) 假设不考虑求解算法的时间成本，若采用分治法求解，请给出算法的基本思想和伪代码。
- (4) 假设采用动态规划求解，请给出算法的伪代码；并分析与分治法相比，动态规划方法为什么能够有效减少求解时间。

